THE UNIVERSITY OF MELBOURNE

SWEN90006-Week 12

Advanced Topics on Security Testing



Dr. Thuan Pham

The University of Melbourne, Australia

Weeks-7-11 Recap

- What are software vulnerabilities?
- What is fuzzing? How does it work?
- Major types of fuzzing: random fuzzing, mutation-based fuzzing, generation-based fuzzing, code coverage-guided fuzzing
- What is symbolic execution?
- Some hybrid fuzzing approaches, including dynamic symbolic execution
- Test oracles for detecting software vulnerabilities

What are the current “hot” and advanced research topics on fuzzing/security testing [1]

1) **Making fuzzing more effective**
   - i.e., discovering *more bugs* and/or *more types* of bugs with fewer *assumptions*
2) **Making fuzzing more efficient**
   - i.e., finding bugs *faster*
3) **Human-Tool teaming for security testing**
   - i.e., leveraging *human domain knowledge* to support fuzzing in an interactive manner



SHONAN meeting in Japan in Sept 2019



Dagstuhl Seminar in Germany in Mar 2023

(1) Making fuzzing more effective

- **To discover more bugs,**
  - Hybrid fuzzing approaches (e.g., dynamic symbolic execution, CGF + DSE, structure-aware fuzzing)
  - …
- **To discover more types of bugs,**
  - *Stateful network protocol fuzzing* (e.g., AFLNet)
  - *Logic/semantic bug detection* (e.g., SQL injection, Excessive Data Exposure)
  - …
- **With fewer assumptions**
  - Working with program binaries, *no source code is required*
  - …

Beyond Well-tested Applications

AFLNET: A Greybox Fuzzer for Network Protocols

Van-Thuan Pham
Monash University
thuan.pham@monash.edu

Marcel Böhme
Monash University
marcel.boehme@monash.edu

Abhik Roychoudhury
National University of Singapore
abhik@comp.nus.edu.sg

***Abstract*—Server fuzzing is difficult. Unlike simple command-line tools, servers feature a massive state space that can be traversed effectively only with well-defined sequences of input messages. Valid sequences are specified in a *protocol*. In this paper, we present AFLNET, the first greybox fuzzer for protocol implementations. Unlike existing protocol fuzzers, AFLNET takes a mutational approach and uses state-feedback to guide the fuzzing process. AFLNET is seeded with a corpus of recorded message exchanges between the server and an actual client. No protocol specification or message grammars are required. AFLNET acts as a client and replays variations of the original sequence of messages sent to the server and retains those variations that were effective at increasing the coverage of the code or state space. To identify the server states that are exercised by a message sequence, AFLNET uses the server's response codes. From this feedback, AFLNET identifies progressive regions in the state space, and systematically steers towards such regions. The case studies with AFLNET on two popular protocol implementations demonstrate a substantial performance boost over the state-of-the-art. AFLNET discovered two new CVEs which are classified as critical (CVSS score CRITICAL 9.8).**

AFLNet Five Years Later: On Coverage-Guided Protocol Fuzzing

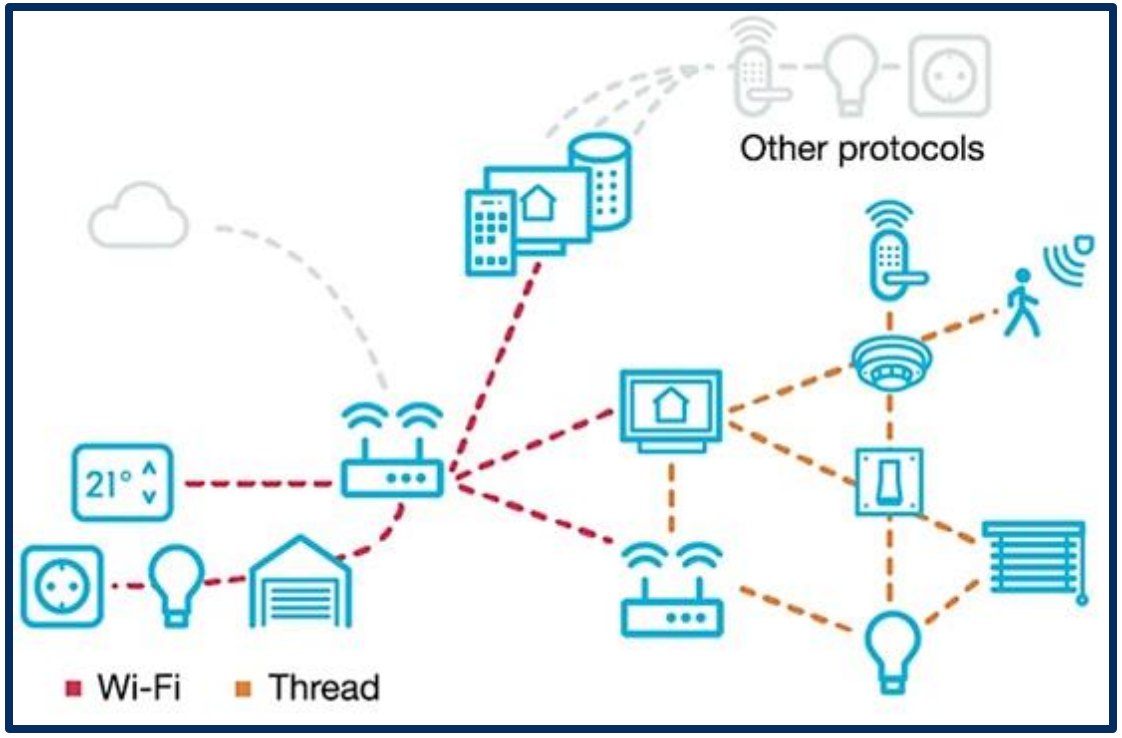
Ruijie Meng, Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury

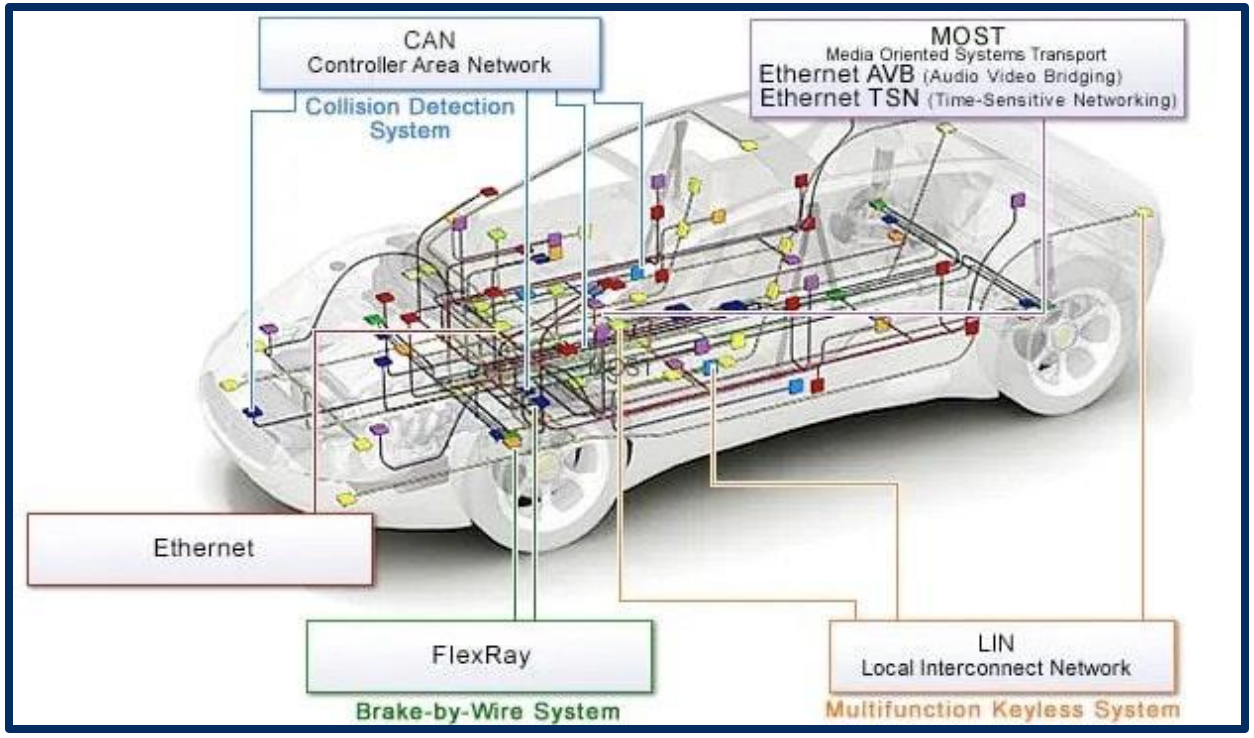
Abstract—Protocol implementations are stateful which makes them difficult to test: Sending the same test input message twice might yield a different response every time. Our proposal to consider a sequence of messages as a seed for coverage-directed greybox fuzzing, to associate each message with the corresponding protocol state, and to maximize the coverage of both the state space and the code was first published in 2020 in a short tool demonstration paper. AFLNet was the first code- and state-coverage-guided protocol fuzzer; it used the response code as an indicator of the current protocol state. Over the past five years, the tool paper has gathered hundreds of citations, the code repository was forked almost 200 times and has seen over thirty pull requests from practitioners and researchers, and our initial proposal has been improved upon in many significant ways. In this paper, we first provide an extended discussion and a full empirical evaluation of the technical contributions of AFLNet and then reflect on the impact that our approach and our tool had in the past five years, on both the research and the practice of protocol fuzzing.

Index Terms—Greybox Fuzzing, Network Protocol Testing, Stateful Fuzzing.

Greybox-Fuzzing for Stateful Network Protocols

Why should we care about the security of network protocols?





*images from Internet (favicon etc)

Fuzzing stateful protocol implementations is challenging

- Server accepts *sequences of messages*
- Server behaviour depends on the *current input & current program state*
  - *message order matters*
  - *Knowing current server state & how to drive the search towards a specific state is important*

220 FTP Server ready
USER foo
331 User foo OK. Password required
PASS foo
230 User logged in, proceed.
MKD demo
257 Directory created.
CWD demo
250 Requested file action okay, completed.
STOR test.txt
150 File status okay
226 Transfer complete
QUIT
221 Goodbye!

A sample FTP session to upload a file (test.txt) to a new folder (demo) on the server

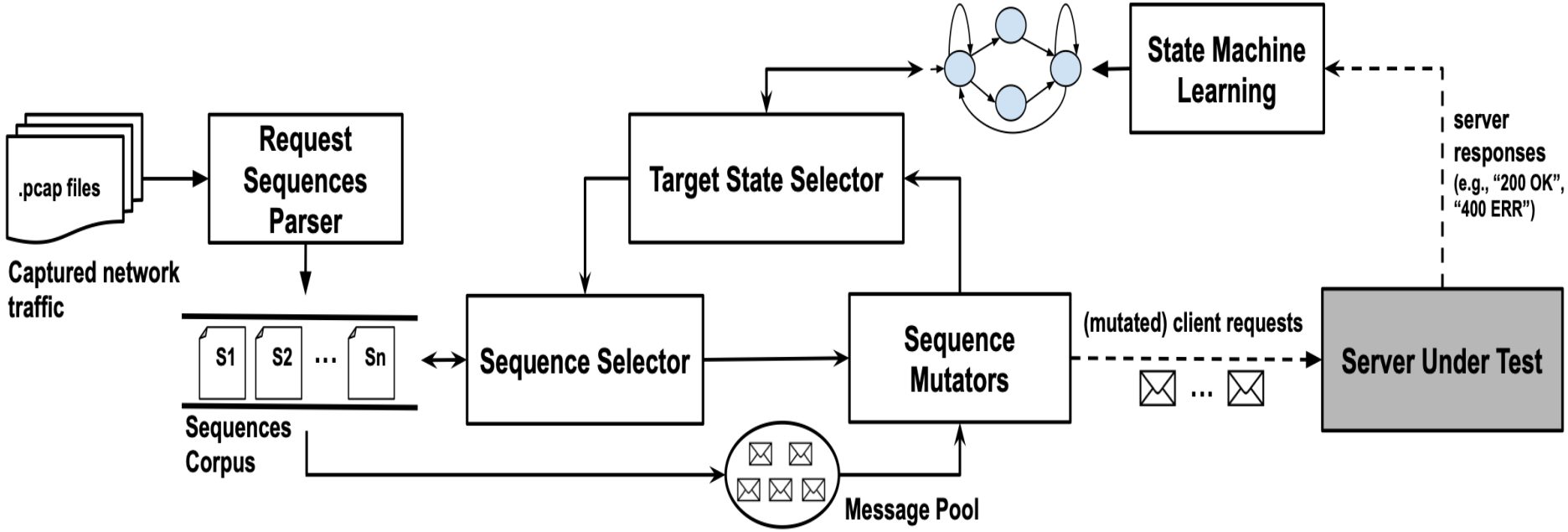
Existing widely-used approaches

1) Stateful Black-box Fuzzing, e.g., Peach, BooFuzz
   - Require manual constructed state machine/model
   - With no/limited feedback
2) Stateless Grey-box Fuzzing: AFL

> *"One of the things that I struggle with is the limitation AFL seems to have, in that it only performs fuzzing with one input (a file). For many systems such as network protocols, it would be useful if fuzzing could be done on a sequence of inputs. This sequence of inputs might be for example messages necessary to complete a handshake in TLS/TCP."*
>
> — Paul (a member of the AFL's user group) [8]

> *"I'm interested in doing something fairly non-traditional and definitely not currently supported by AFL. I would like to perform fuzzing of a large and complex external server that cannot easily be stripped down into small test cases."*
>
> — Tim Newsham (a member of the AFL's user group) [8]

Architecture of AFLNet

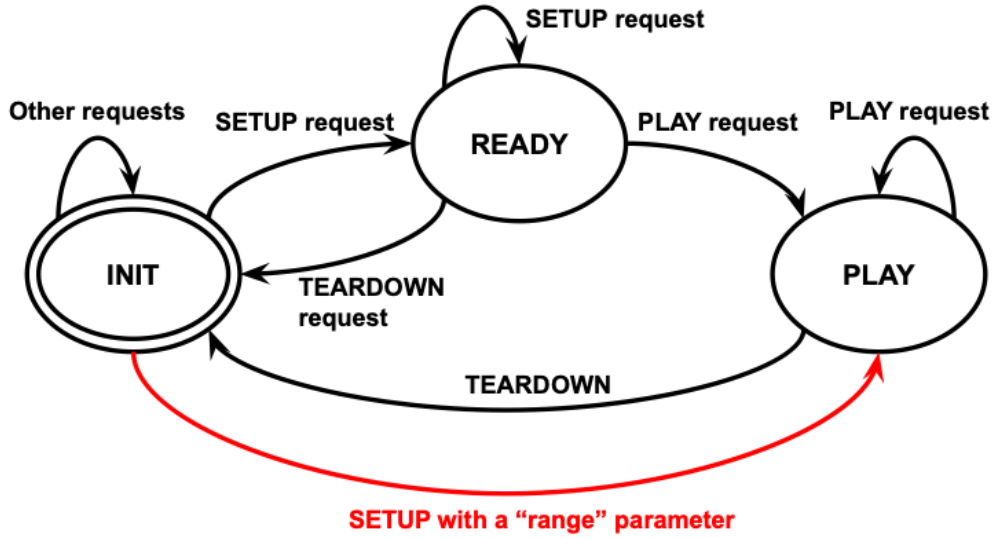


AFLNet takes a ***mutational approach*** and uses ***state-feedback*** to *dynamically* construct ***protocol state machine*** and guide the fuzzing process, together with code coverage feedback.

- AFLNet acts as a client.
- AFLNet is seeded with a corpus of recorded message exchanges.
- AFLNet monitors server behaviours (e.g., code coverage) and its responses

Automated State Machine Inference

- Time consuming
- Require domain knowledge
- Implemented protocol could be different from the standard specification



Manual & static approach

- Not time consuming
- Capture the exact implemented protocol

Automatic & dynamic approach

220 FTP Server ready

USER foo

331 User foo OK. Password required

PASS foo

230 User logged in, proceed.

MKD demo

257 Directory created.

CWD demo

250 Requested file action okay, completed.

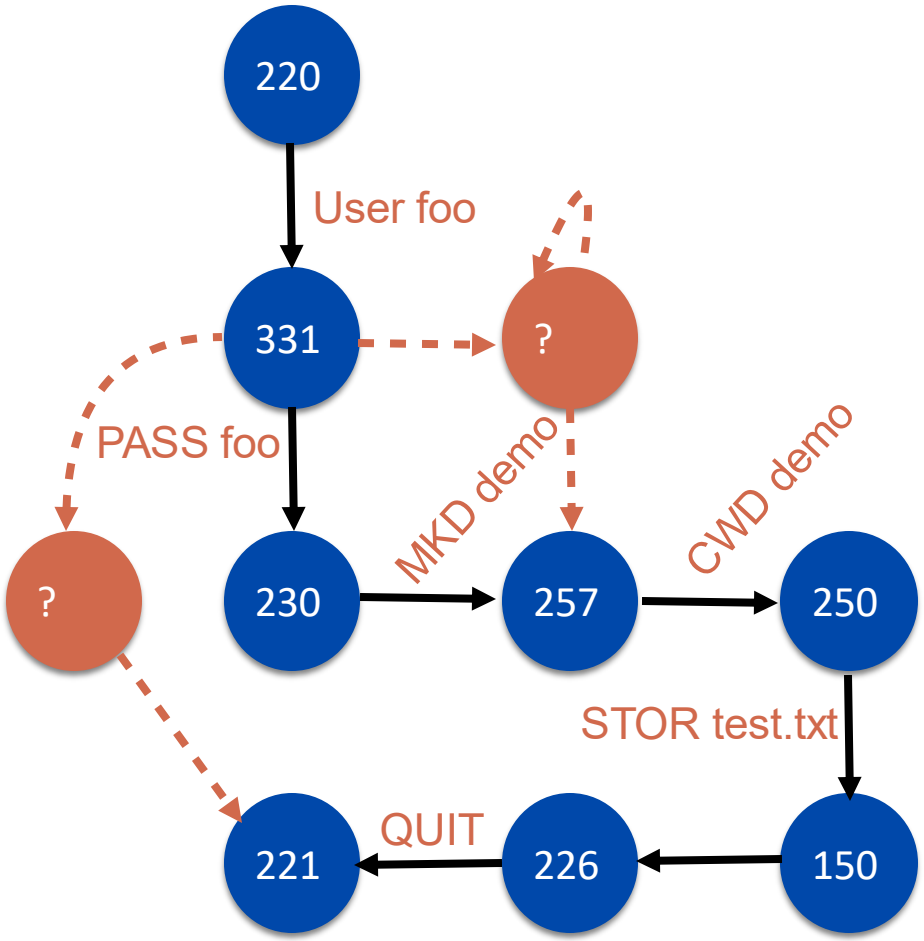
STOR test.txt

150 File status okay

226 Transfer complete

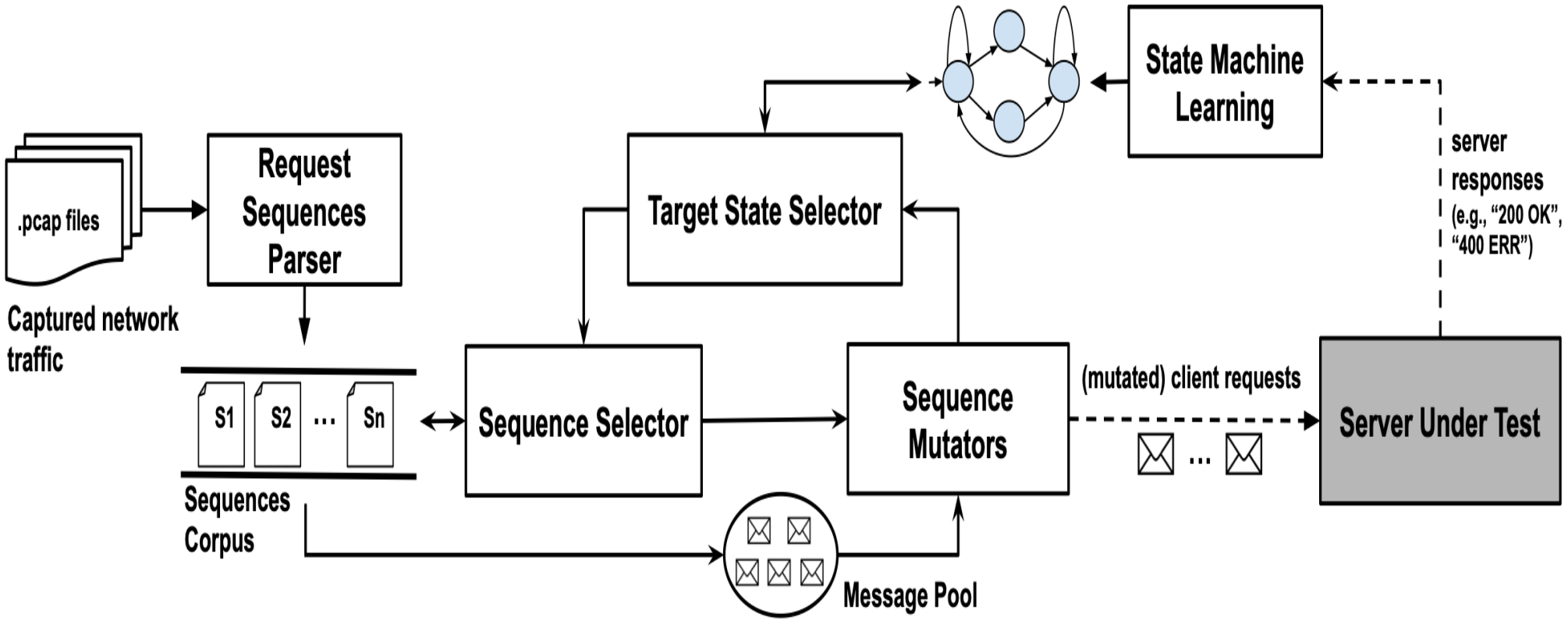
QUIT

221 Goodbye!



AFLNet workflow

Prioritise *"progressive" states* that contribute more towards increased code coverage.

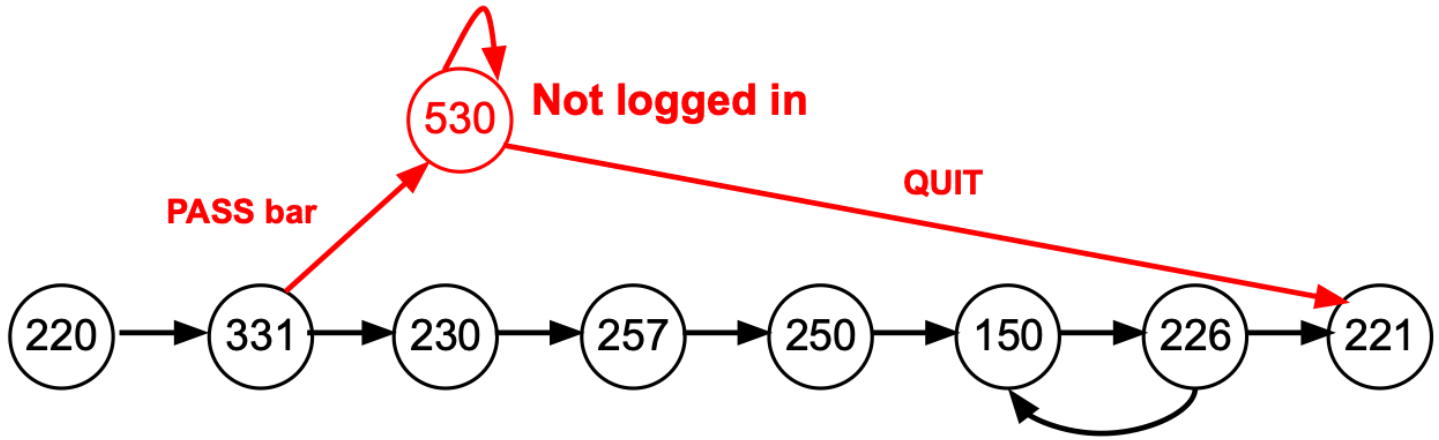


Original message sequence (i.e., seed input)



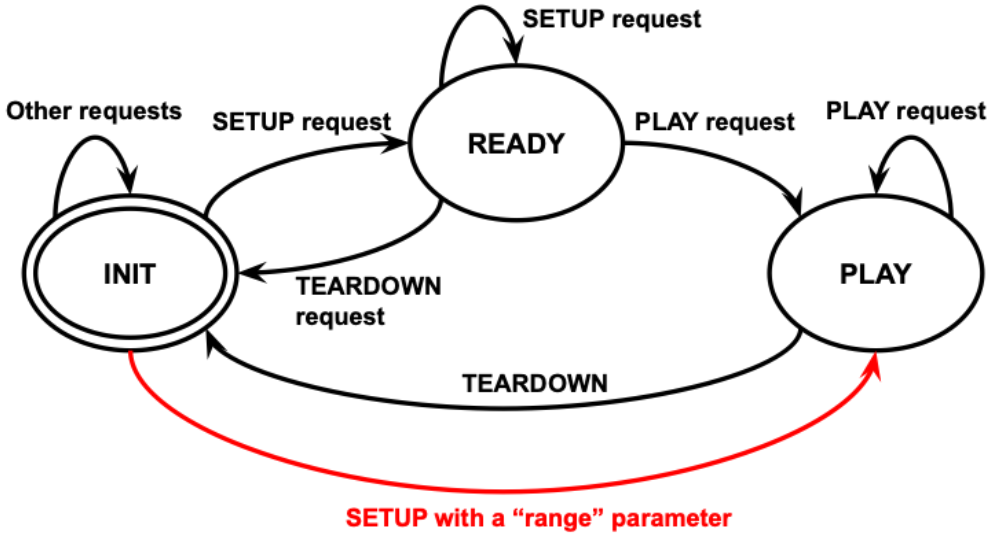
And then, state 331 (User OK) is targeted





inferred state machine

CVE-2019-7314 - liblivemedia in Live555 before 2019.02.03 mishandles the termination of an RTSP stream after RTP/RTCP-over-RTSP has been set up, which could lead to a Use-After-Free error that causes the RTSP server to crash (Segmentation fault) or possibly have unspecified other impact.





Alban Lecocq @skeetmtp · 13 Jan

I'm using Afl to find "packet of death" for 3 years, but never manage to detect statefull bug with it. Indeed there little litterature on the subject. Can't wait to read more details on #AFLNet

AFLNet keeps evolving

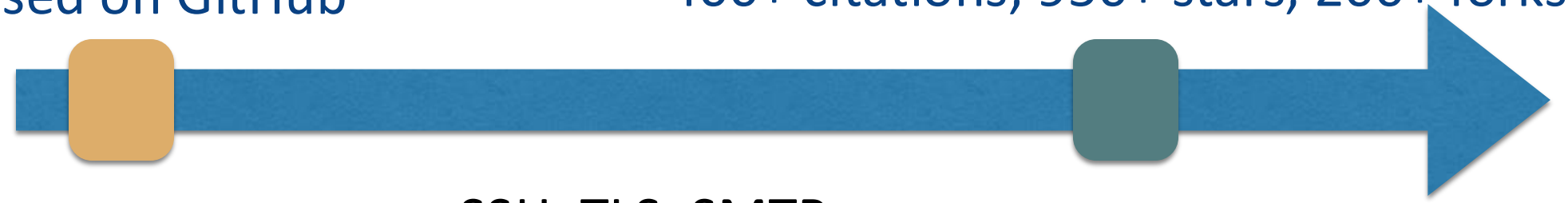
https://github.com/aflnet/aflnet

Mar 2020

Released on GitHub

Sept 2025

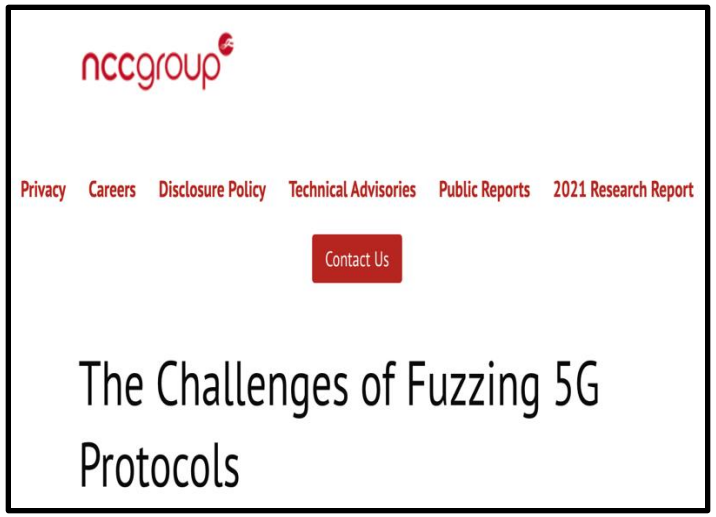
400+ citations, 950+ stars, 200+ forks



FTP, RTSP

+ SSH, TLS, SMTP,

DTLS, DICOM, DNS, SIP, MODBUS …







Impacts from our work

- AFLNet has ***pioneered coverage-guided fuzzing for stateful network protocols***
- AFLNet has discovered ***several CVEs***
- AFLNet is considered a viable open-source option for automotive cybersecurity under the ***ISO/SAE 21434 standard***
- Received ***funding from an industry partner to*** to integrate AFLNet into their SDK
- ***Several companies have tried and extended*** AFLNet
- AFLNet ***has been studied*** at institutions including NUS, CMU, UoM

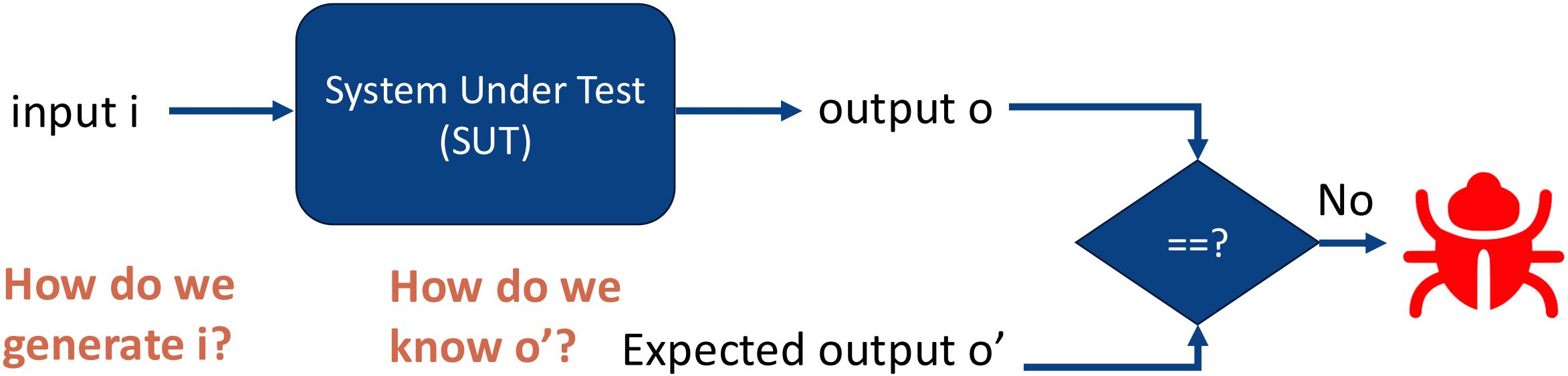
On-going/Future Work

LLM-guided network protocol fuzzing

- Leverage LLM to infer the state machine from the spec & code to support AFLNet
- Leverage LLM to analyze logs for bug detection
- …

Beyond Crash Oracles

Software Testing – Test Oracle Problem

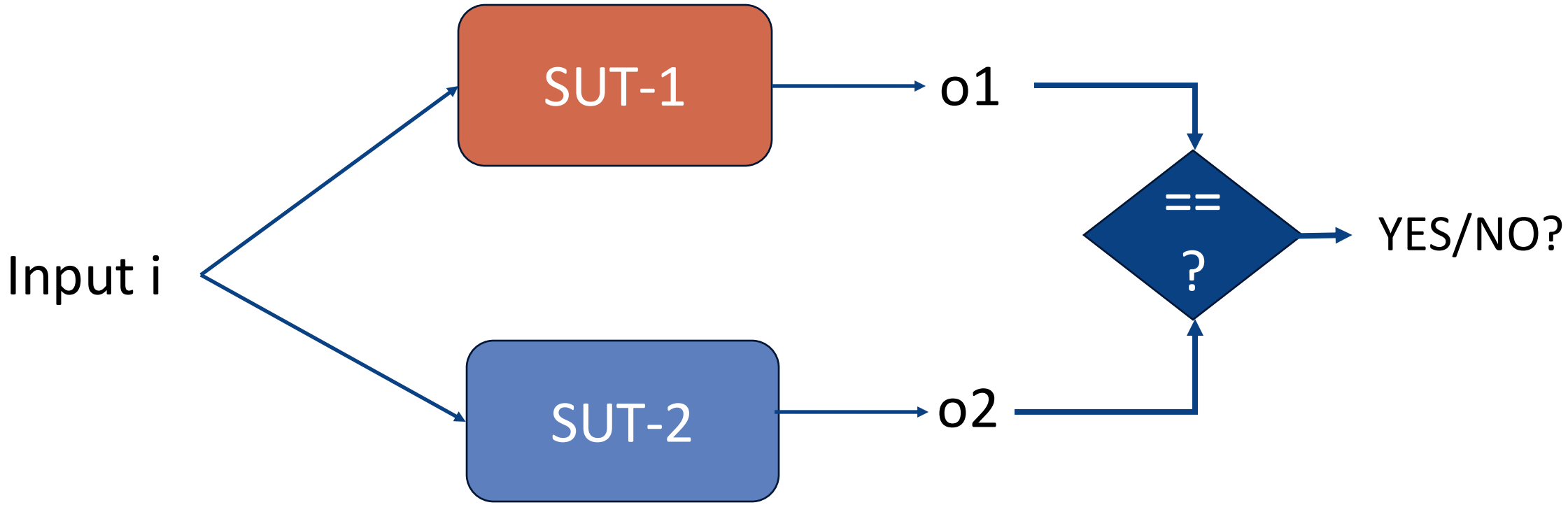


"Given an input for a system, the challenge of distinguishing the corresponding desired, correct behaviour from potentially incorrect behavior is called the test oracle problem" **[Earl et al. 2015]**

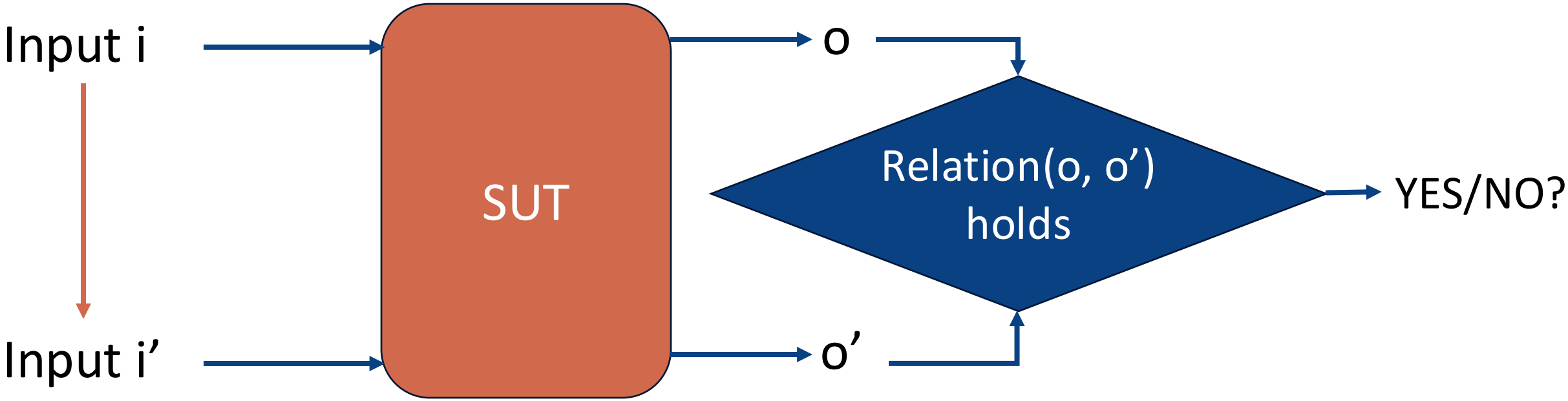
Common solutions

- Human Oracles
- “Explicit” Oracles
  - Crashes (e.g., SEGV, ABORT including sanitizer aborts)
  - Hangs
- Differential Testing [William M. McKeeman]
- Metamorphic Testing [T.Y. Chen et al.]

Differential testing



Metamorphic testing



How would you identify metamorphic relations?

Q-1) What is the System/Component-Under Test?

Q-2) What transformations can be applied to the inputs? (e.g., deletion, semantic-preserving transformations, combination, addition)

Q-3) Does it exist any relation between o and o’ that should hold on all pairs (i, i’) where i’ = transformation(i)?

Examples: identify metamorphic relations

Ex 1: to test a shortest path finding algorithm implementation (e.g., Dijkstra algorithm)?

Ex 2: to test an CNN-based image recognition system (e.g., used in autonomous cars)?



EDEFuzz: A Web API Fuzzer for Excessive Data Exposures



Lianglu Pan, Shaanan Cohney, Toby Murray, Van-Thuan Pham
lianglup@student.unimelb.edu.au,shaanan@cohney.info,{toby.murray,thuan.pham}@unimelb.edu.au
The University of Melbourne, Melbourne, Australia

ABSTRACT

APIs often transmit far more data to client applications than they need, and in the context of web applications, often do so over public channels. This issue, termed *Excessive Data Exposure* (EDE), was OWASP's third most significant API vulnerability of 2019. However, there are few automated tools—either in research or industry—to effectively find and remediate such issues. This is unsurprising as the problem lacks an explicit test oracle: the vulnerability does not manifest through explicit abnormal behaviours (e.g., program crashes or memory access violations).

In this work, we develop a metamorphic relation to tackle that challenge and build the first fuzzing tool—that we call EDEFuzz—to systematically detect EDEs. EDEFuzz can significantly reduce false negatives that occur during manual inspection and ad-hoc text-matching techniques, the current most-used approaches.

We tested EDEFuzz against the sixty-nine applicable targets from the Alexa Top-200 and found 33,365 potential leaks—illustrating our tool's broad applicability and scalability. In a more-tightly controlled experiment of eight popular websites in Australia, EDEFuzz achieved a high true positive rate of 98.65% with minimal configuration, illustrating our tool's accuracy and efficiency.

> *"Automatic tools usually can't detect this type of vulnerability because it's hard to differentiate between legitimate data returned from the API, and sensitive data that should not be returned without a deep understanding of the application."*
>
> — The Open Web Application Security Project (OWASP)

> *"This vulnerability is so prevalent (place 3 in the top 10) because it's easy to miss. Automation is near useless here because robots can not tell what data should not be served to the user without telling them exactly how the application should work. This is bad because API's are often implemented in a generic way, returning all data and expecting the front-end to filter it out."*
>
> — Wallarm End-to-End API Security Solution

Figure 1: Industry views on the EDEs. These indicate the prevalence of EDEs and limitations of existing detection tools

We start with a definition: an API is vulnerable to EDE if it exposes meaningfully more data than what the client legitimately needs [1].

Consider a simple example of an online storefront. When a user views the page for a specific product, an API call may be made to

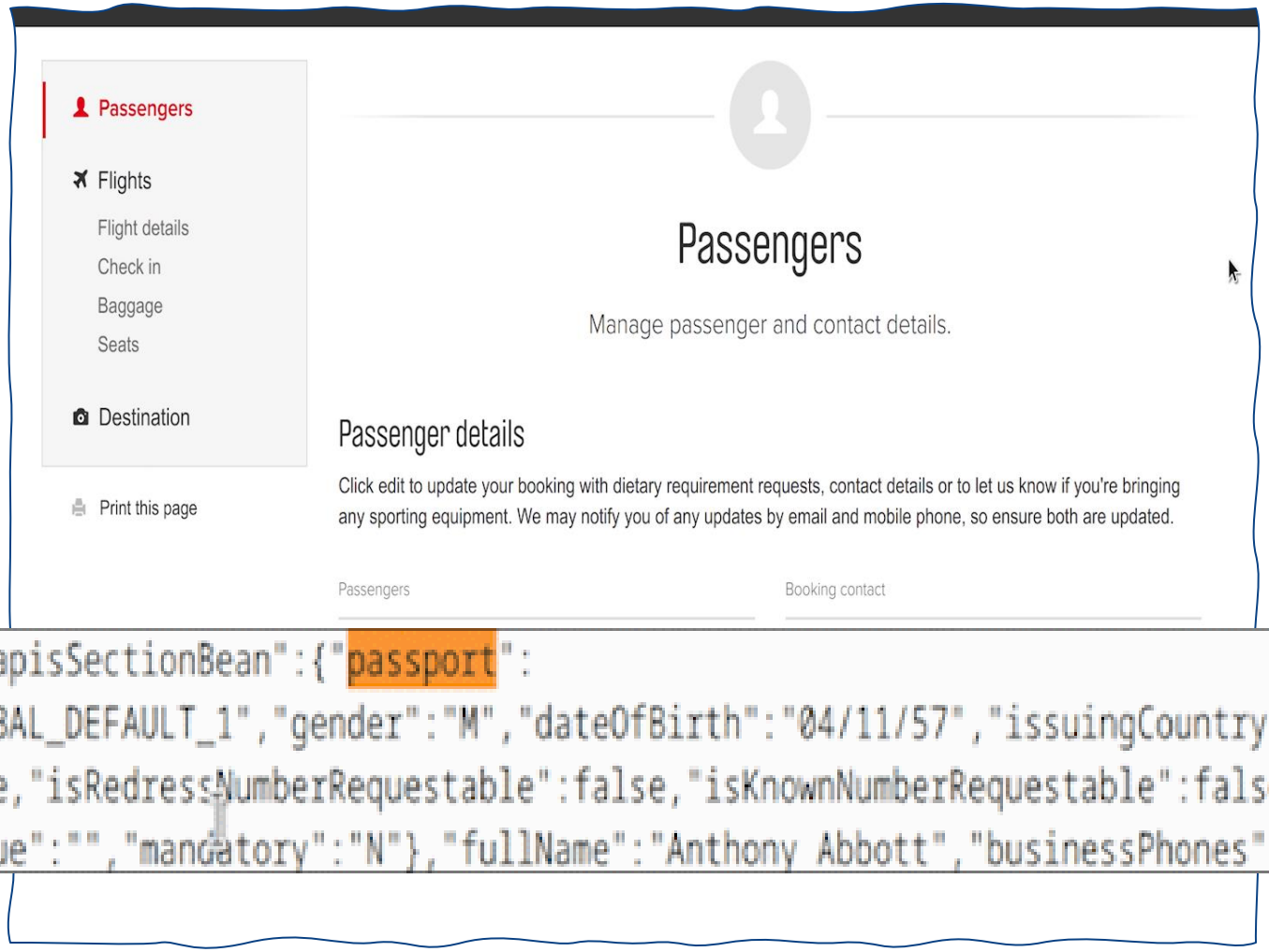
Detecting Excessive Data Exposures in Web APIs

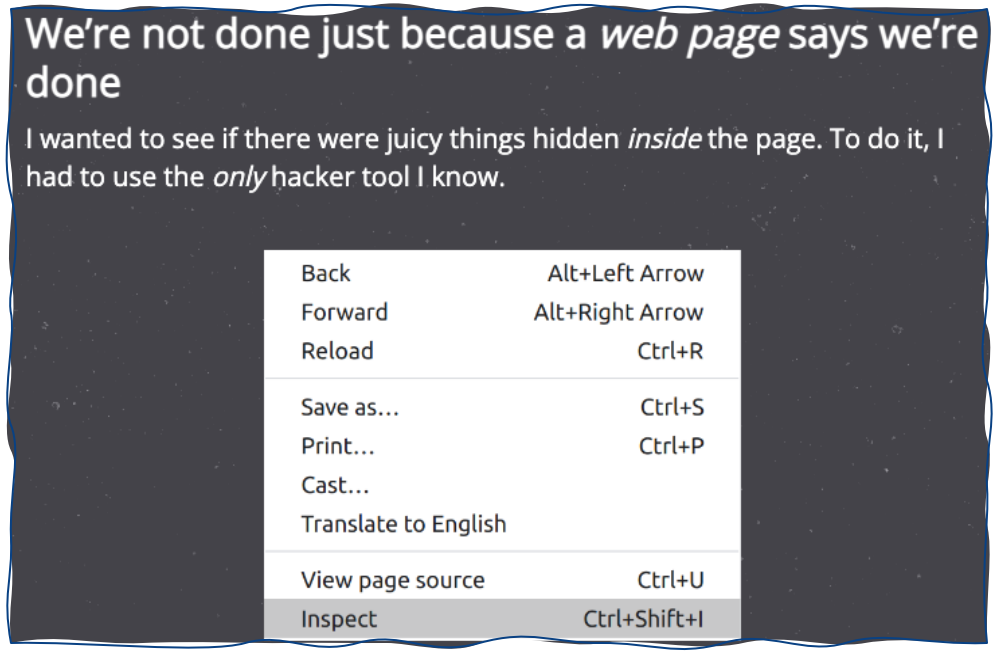
Why should we care about excessive data exposures/data leak?

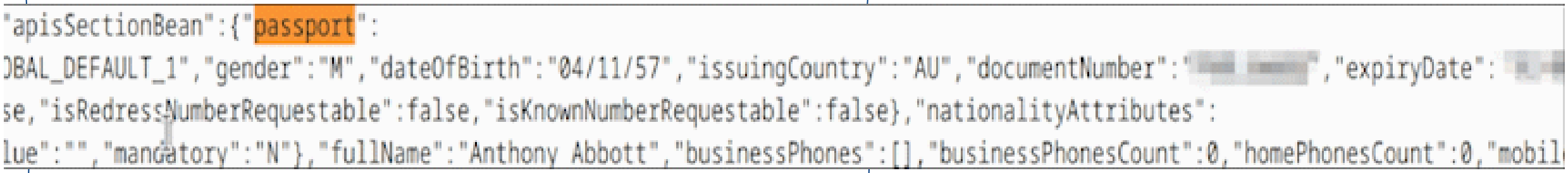




What happened?







Details: https://mango.pdf.zone/finding-former-australian-prime-minister-tony-abbotts-passport-number-on-instagram

Excessive Data Exposures & Its prevalence

Ranked 3rd in OWASP 2019 TOP-10 critical vulnerabilities in APIs

- The API (e.g., Web API) returns full data objects as they are stored in the backend database.
- The client application filters the responses and only shows the data that the users really need to see.
- Attackers call the API directly and get also the sensitive data that the UI would filter out.

> *"Automatic tools usually can't detect this type of vulnerability because it's hard to differentiate between legitimate data returned from the API, and sensitive data that should not be returned without a deep understanding of the application."*
>
> — The Open Web Application Security Project (OWASP)

> *"This vulnerability is so prevalent (place 3 in the top 10) because it's easy to miss. Automation is near useless here because robots can not tell what data should not be served to the user without telling them exactly how the application should work. This is bad because API's are often implemented in a generic way, returning all data and expecting the front-end to filter it out."*
>
> — Wallarm End-to-End API Security Solution

Figure 1: Industry views on the EDEs. These indicate the prevalence of EDEs and limitations of existing detection tools

Manual Detection?

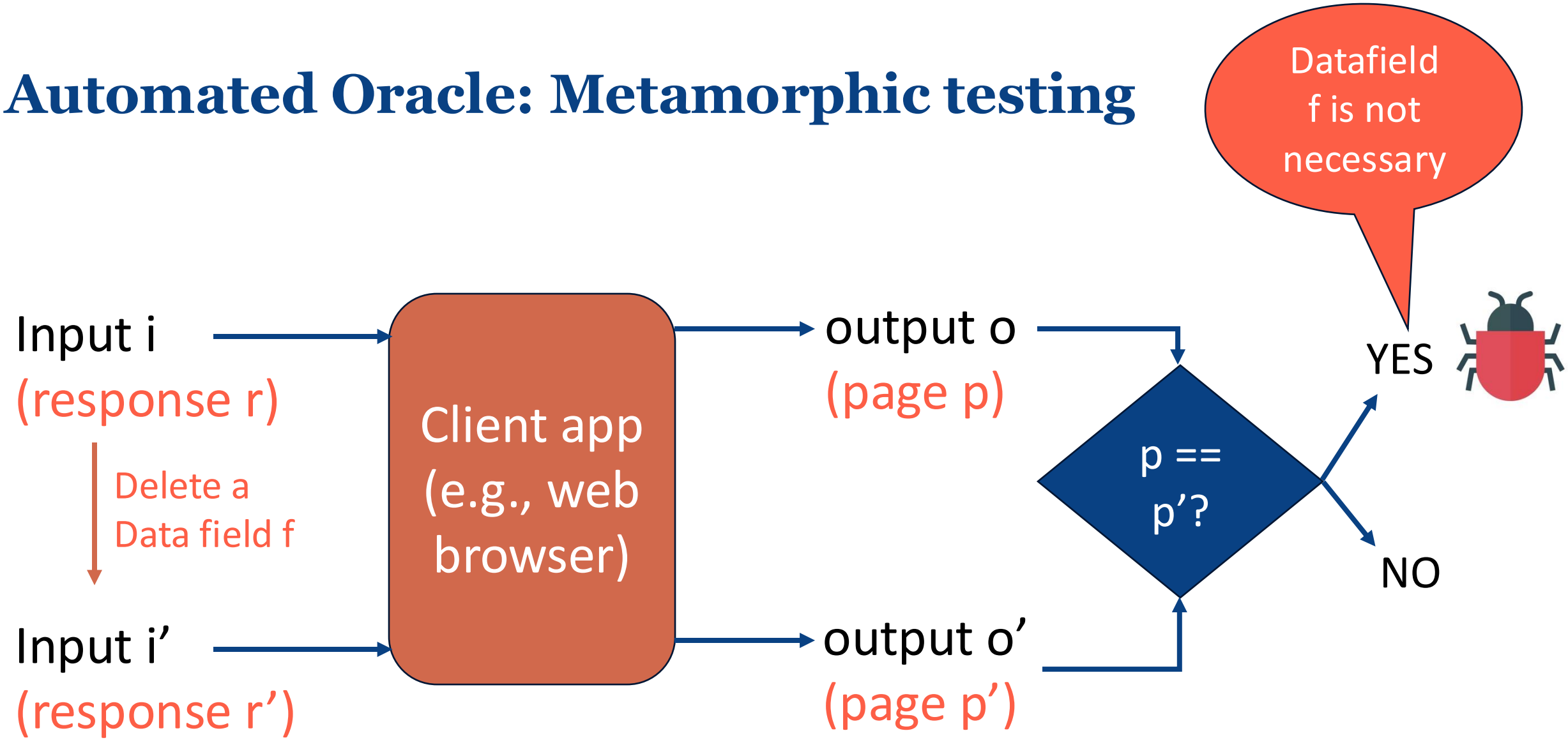
{"id":279980,"vin":"WVWZZZCDZNW001240","model_code":"CD13NS\/22","car_configurator_model_code":"CD13NS-GPJ3PJ3-GPLAPLA-GPRDPRD-GPZFPZF-GWA3WA3-GWCRWCR-GWC4WC4","brand":"Volkswagen","sub_brand":"PV","model_year":"2022","is_freestock":false,"carline_model":"Golf","model_variant":"110TSI Life","model_family":"Golf","body_type":"Hatch","body_type_sort_order":1,"common_color":"Yellow","vehicle_type":"New","transmission":"Auto","fuel":"Petrol","driven_wheels":"FWD","dealer_code":"30281","dealer_state":"VIC","dealer_postcode":"3875","mrdp":"46031","payload":{"vehicle_details":{"carline_model":"Golf","model_variant":"110TSI Life","model_code":"CD13NS\/22","car_configurator_model_code":"CD13NS-GPJ3PJ3-GPLAPLA-GPRDPRD-GPZFPZF-GWA3WA3-GWCRWCR-GWC4WC4","brochure_code":"GOLF","body_shape":"Hatch","model_year":"2022","engine_capacity":"1.4 TSI","transmission":"Auto","transmission_desc":"8-Speed Automatic","fuel":"Petrol","driven_wheels":"FWD","model_family":"Golf","vin":"WVWZZZCDZNW001240","list_in_stock":true,"common_color":"Yellow","color_code":"C1C1","vehicle_type":"New","vehicle_options":"MP,PM,PZC","customer_order_status":"5","is_freestock":false,"brand":"Volkswagen","sub_brand":"PV","demo_kilometers_driven":null,"demo_kilometer_updated":null},"default_settings":{"default_finance_term":"48","default_finance_pa_comparison_rate":"8.82","default_finance_deposit":"10","default_finance_allowance":"60000","default_driveaway_disclaimer":"a4r2e0000007IEWAA2","default_finance_disclaimer":"a4r2e0000007I9kAAE","default_driveaway_legend_icon":"~"},"images":{"new":{"hero_image_listing_page":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/right","hero_image_detail_page":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/front","gallery":{"front":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/front","back":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/back","right":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/right","left":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/left"}},"demo":{"hero_image_listing_page":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/right","hero_image_detail_page":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/front","gallery":{"image0":"https:\/\/vga-images.herokuapp.com\/VICCI\/stock\/image\/WVWZZZCDZNW001240\/front","image1":null,"image2":null,"image3":null,"image4":null,"image5":null}}},"dealer_details":{"dealer_code":"30281","dealer_name":"THE BIG GARAGE VOLKSWAGEN","dealer_state":"VIC","dealer_postcode":"3875","info":{"dealerName":"THE BIG GARAGE VOLKSWAGEN","dealerCode":"30281","locality":"BAIRNSDALE","dealerAddress":"628 PRINCES HIGHWAY, BAIRNSDALE","dealerState":"VIC","dealerPostcode":"3875","webKey":"","UsedCarDealer":"False","PartsAvailable":"True","PartsPostCode":"3875","SalesPostCode":"3875","ServicePostCode":"3875","PartsStateCode":"VIC","SalesStateCode":"VIC","ServiceStateCode":"VIC","PartsSuberbCode":"BAIRNSDALE","SalesSuberbCode":"BAIRNSDALE","ServiceSuberbCode":"BAIRNSDALE","PartsEmailCode":"","SalesEmailCode":"","ServiceEmailCode":"","PartsPhoneCode":"(03) 5152 4131","SalesPhoneCode":"(03) 5152 4131","ServicePhoneCode":"(03) 5152 4131","PartsStreetCode":"628 PRINCES HIGHWAY","SalesStreetCode":"628 PRINCES HIGHWAY","ServiceStreetCode":"628 PRINCES HIGHWAY","PartsFaxCode":"","SalesFaxCode":"","ServiceFaxCode":"","FinanceEmail":"","ServiceDealer":"True","phone":"0351525313","tradingHours":[{"name":"Sunday","isclosed":true,"startTime":"09:00 AM","endTime":"05:00 PM"},{"name":"Monday","isclosed":false,"startTime":"08:30 AM","endTime":"05:30 PM"},{"name":"Tuesday","isclosed":false,"startTime":"08:30 AM","endTime":"05:30 PM"},{"name":"Wednesday","isclosed":false,"startTime":"08:30 AM","endTime":"05:30 PM"},{"name":"Thursday","isclosed":false,"startTime":"08:30 AM","endTime":"05:30 PM"},{"name":"Friday","isclosed":false,"startTime":"08:30 AM","endTime":"05:30 PM"},{"name":"Saturday","isclosed":false,"startTime":"09:00 AM","endTime":"12:00 PM"}],"publicHolidays":["2018-11-06","2019-03-11","2019-01-28","2019-04-19","2019-04-21","2019-04-22"],"mapLocation":"","DealerOpTiming":[{"Type":"Parts","operatingTime":"Mon:0830-1700,Tue:0830-1700,Wed:0830-1700,Thu:0830-1700,Fri:0830-1700"},{"Type":"Service","operatingTime":"Mon:0830-1700,Tue:0830-1700,Wed:0830-1700,Thu:0830-1700,Fri:0830-1700"}]}},"tile_info":{"background_color":"#e4d330","font_color":"#000"},"careplanData":{"model":"CD13NS\/22","careplan_model_code":"CD13NS\/22_CARE_PLAN_5","careplan_name":"5-Year Care Plan","term":"60","price_inc_gst":"2400","price_exc_gst":"2181.82","Policy_Type":"Service Pack"},"optional_packages":[{"name":"Metallic Paint","code":"GMPMP","price":"650"},{"name":"Premium Metallic Paint","code":"GPMPM","price":"900"},{"name":"Sound & Vision Package","code":"GPZCPZC","price":"1500"}],"optional_packages_comb_string":"Metallic Paint, Premium Metallic Paint, Sound & Vision Package","banner":{"banner_heading":"MY22 Finance Offer","sub_headings":["Free 3 Year\/45,000 KM Care Plan\u2020"],"application":"Offer;Stock","vehicle_type":"New","banner_end_date":"2022-06-30","banner_start_date":"2022-04-01"},"pcsOfferIds":["a2E9q0000005AWvEAM"],"offers":{"Driveaway":{"offer_name":"Driveaway~","offer_heading_1":null,"offer_price":"0","applications":"Offer;Stock","start_date":"2022-04-01","end_date":"2022-06-30","legend_icon":null,"term":null,"pa_comparison_rate":null,"allowance":null,"deposit_required":null,"is_deposit_in_cash":false,"offer_type":"Driveaway","disclaimer":"a4r9q00000006n8AAA","disclaimer1":null,"disclaimer2":null}},"discalimers":[{"id":"a4r9q00000006n8AAA","name":"D-0077","legend_icon":null,"description":"PV MY22 Golf Driveaway Disclaimer","disclaimer":"~Manufacturer's recommended driveaway price (MRDP) for new MY22 vehicles in white sold and delivered on or after 01\/06\/2022."}],"dealer_comment":null},"last_modified_date":"2022-04-13 14:05:27","version":238,"status":1,"inserted_time":"2022-04-15 02:37:21","updated_time":"2022-04-15 02:37:21","did":"30281","ordering":23}

How can one automatically detect if a web API exposes more data than it should?

- The question is related to the famous ***test oracle problem***.
- We address this challenge with the following key insight:
  - ➢ Data returned from an API endpoint is more likely excessive if it has no impact on the content displayed to a user.
  - ➢ This is a metamorphic relation

if a field in a JSON response is not needed,
the rendered webpage should not change if the field is deleted.

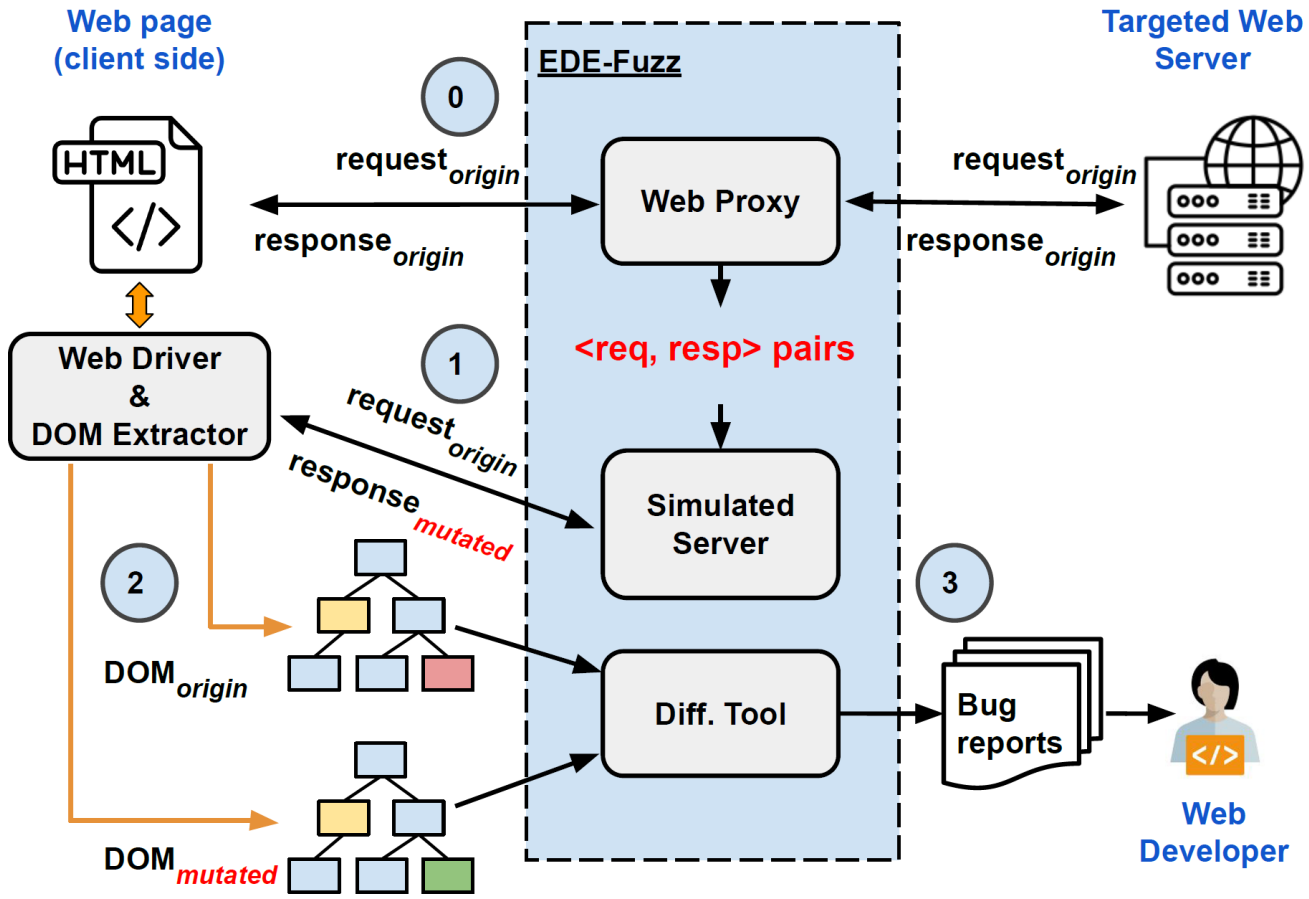
Automated Oracle: Metamorphic testing



Workflow of EDEFuzz

Three simple steps:

1) Save a copy of the rendered webpage after the API call
2) Try deleting *each of the fields* in the response
3) Check if the newly rendered page is the same as the saved copy. No change? The data was unnecessary



Results & Discussions

Target	Data fields	Reported	Confirmed	TP	Preparation (min)	Execution (min)	Classification (min)	Sensitive	Non-sensitive
COMPANY-A	189	124	124	100.00%	10	11	5	0	124
COMPANY-B	18	16	14	87.50%	20	2	2	2	12
COMPANY-C	2600	2580	2504	97.05%	5	306	3	104	2400
COMPANY-D	545	506	479	94.66%	15	43	10	9	470
COMPANY-E	4249	4147	4127	99.52%	10	755	15	0	4127
COMPANY-F	778	749	749	100.00%	15	103	5	0	749
COMPANY-G	120	100	100	100.00%	5	12	3	0	100
COMPANY-H	1465	1066	1066	100.00%	15	79	20	19	1047

Table 2: Summary statistics from the Australian sites. Data fields reports the total number of fields contained in the API response of each target, Reported is the number of fields flagged by EDEFuzz as excessive; Confirmed is the number of fields manually confirmed to be excessive, i.e. true positives, TP. The time taken to configure EDEFuzz for each target is reported in Preparation, as is the duration of test execution (Duration) and the human effort required to manually classify the flagged fields as sensitive or not (Classification), all measured in minutes. We also report (Sensitive) the number of fields we classified as containing sensitive data, after manual inspection.

Impacts from our work

- Discovered and reported several data leakage issues to Australian companies. One CVE (CVE-2025-51498) for a vulnerability in Strapi – one of the leading headless CMS systems (~70k stars on GitHub).
- EDEFuzz won a distinguished paper ***award*** at the flagship conference ICSE'24
- An Australian provisional ***patent*** for EDEFuzz
- Received ***Letters of Intent/interest*** from several companies

Ongoing & Future Work

- LLM-guided Web API fuzzing
  - Leverage LLM to support EDEFuzz, reducing human effort to check the results
  - …

Reading recommendation

Toss a Fault to Your Witcher: Applying Grey-box Coverage-Guided Mutational Fuzzing to Detect SQL and Command Injection Vulnerabilities

Erik Trick
Giovanni Vigna†, Christopher Kru

Automated Black-box Testing of Mass Assignment Vulnerabilities in RESTful APIs

Davide Corradini*, Michele Pasqua† and Mariano Ceccato‡
Department of Computer Science

FUZZORIGIN: Detecting UXSS vulnerabilities in Browsers through Origin Fuzzing

Sunwoo Kim*
Samsung Research
sunwoo28.kim@samsung.com

Young Min Kim
Seoul National University
ym.kim@snu.ac.kr

Jaewon Hur
Seoul National University
hurjaewon@snu.ac.kr

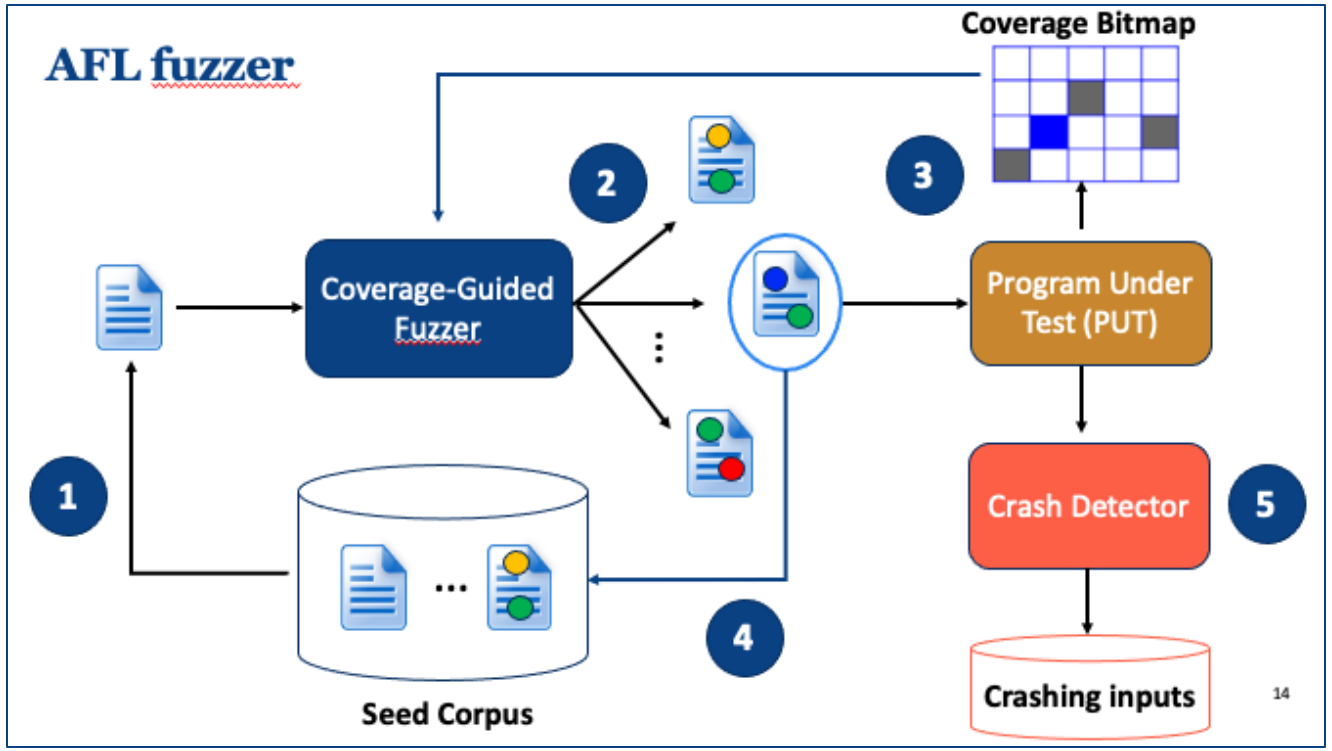
Suhwan Song
Seoul National University
sshkeb96@snu.ac.kr

Gwangmu Lee†
EPFL
gwangmu.lee@epfl.ch

Byoungyoung Lee‡
Seoul National University
byoungyoung@snu.ac.kr

With fewer assumptions

Fuzzing program binaries – no source code is required



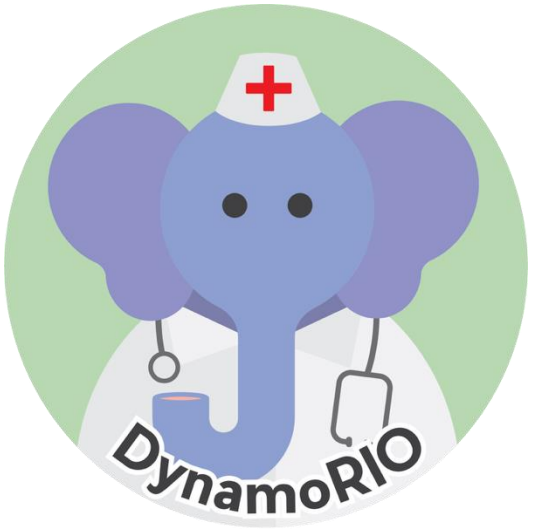
How do we collect code coverage information?

- Using Dynamic Binary Instrumentation (e.g., DynamoRIO, Intel PIN tool)
- Using hardware tracing with Intel PT
- Using Static Binary Instrumentation

Dynamic instrumentation with DynamoRIO

Enable analyses on program binaries by “hooking” at the assembly (machine language) level

- DynamoRIO supports event callbacks (e.g., before a branching instruction is executed, a function is called)
- The fuzzer (e.g., WinAFL) captures events and instrument/insert code into the assembly



WinAFL

https://github.com/googleprojectzero/winafl

What's next?

Play with some fuzzing tool(s) and try to catch real bugs. You might get *bug bounties* if you can find security vulnerabilities in widely-used software libraries/systems!!



Fuzzing 101: https://github.com/antonio-morales/Fuzzing101

Are you interested in doing fuzzing research? I am hiring!

- 2 Master's students for MSC-100 projects, working on IoT fuzzing to secure the Matter protocol implementation with AFLNet
  - About the Matter protocol: https://csa-iot.org/all-solutions/matter/
  - Successful candidates will ***get paid*** and have a great opportunity to collaborate with engineers from big-tech companies.
- You have other ideas? Let's schedule a chat.

End of Semester Survey



References

[1] Boehme, M., Cadar, C., & Roychoudhury, A. (2021). Fuzzing: Challenges and Reflections. *IEEE Softw.*, *38*(3), 79-86.

[2] Pham, V. T., Böhme, M., & Roychoudhury, A. (2020, October). AFLNet: a greybox fuzzer for network protocols. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)* (pp. 460-465). IEEE.

[3] Pan, L., Cohney, S., Murray, T., & Pham, V. T. (2024, February). EDEFuzz: A Web API Fuzzer for Excessive Data Exposures. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering* (pp. 1-12).